

Baseino savitarnos ir užimtumo valdymo sistema

Kursinio darbo I dalis: projektavimo dokumentas

1. Problema ir idėja

Sistema vienu sakiniu: Sistema skirta savarankiškai įsigyti baseino bilietą, gauti QR kodą ir matyti baseino užimtumą realiu laiku.

Problema ir dabartinis procesas: Baseino lankytojai nemato, kiek yra laisvų vietų. Atvykę turi klausti kasoje, o esant dideliame užimtume susidaro eilės. Lankytojus aptarnauja vienas darbuotojas. Jam tenka parduoti bilietus ir kartoti informaciją apie užimtumą, kainas bei lankymosi sąlygas. Dėl to lankytojai ilgiau laukia.

Nauda: Lankytojai galės geriau planuoti laiką ir matyti, kada geriausia apsilankyti baseine, taip išsaugojant klientų skaičių. Sistema automatiškai apskaičiuos kainą bei priemoną ir neleis viršyti nustatytos talpos.

Naudotojai: Klientas naudosis savitarna ir bilieto QR kodu. Darbuotojas patvirtins priemokos apmokėjimą, jei klientas viršys apmokėtą laiką.

Prielaidos: Baseino komplekse yra 25 vietos vyrams ir 25 vietos moterims, iš viso - 50. Vienas bilietas skirtas vienam žmogui ir vienam apsilankymui iš karto po bilieto įsigijimo. Klientas pats pasirenka vyrų arba moterų vietų grupę, dokumentai netikrinami. Pirčių, vonių ir baseino vietos atskirai neskaičiuojamos. Pavyzdinės kainos: 60 min. - 6€, 120 min. - 10€, o už kiekvienas pradėtas papildomas 15 min. mokama 2€. Šios kainos skirtos prototipui ir gali būti keičiamos.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Pagrindinis modulis ar pagalbinė funkcija
Bilieto įsigijimas	Pasirinkti grupę ir trukmę, matyti kainą, imituoti mokėjimą ir gauti QR kodą.	Pagrindinis modulis
Įėjimas ir išėjimas	Pateikti kodą. Sistema tikrins būseną ir registruos apsilankymo pradžią bei pabaigą.	Pagrindinis modulis
Priemokos apskaičiavimas	Klientas matys viršytą laiką ir priemoną. Darbuotojas galės patvirtinti apmokėjimą kasoje.	Pagrindinis modulis
Užimtumo ekranas	Matyti, kiek lankytojų yra viduje, kiek vietų rezervuota ir kiek liko laisvų vietų vyrams bei moterims.	Pagalbinė funkcija

Į kursinio darbo apimtį neįeina: Tikri mokėjimai ir grąžinimai, fizinių vartų ar skaitytuvų integracija, visas administratoriaus valdymo langas, klientų paskyros, ateities rezervacijos, abonementai ir atskirų zonų valdymas. Prototipe mokėjimas ir nuskaitymas imituojami mygtukais bei kodo įvedimu. Sugeneruotas QR kodas vaizduoja tą patį bilieto identifikatorių.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Apsilankymo valdymas. Modulis apima bilieto išdavimą, vietos rezervavimą, kodo tikrinimą, laiko apskaitą ir apsilankymo užbaigimą.

Logika, kurią reikės projektuoti ir testuoti: Reikės tikrinti laisvas vietas, valdyti bilietų būsenas ir skaičiuoti priemoną. Taip pat reikės patikrinti, kas vyksta dviem klientams perkant vienu metu. Bilieto būsenos: laukiama įėjimo, aktyvus, laukiama priemokos, užbaigtas ir pasibaigęs. Kol priemoka nesumokėta ir išėjimas neužregistruotas, lankytojas skaičiuojamas kaip esantis viduje.

Įvestis: Pirkimui - vyras arba moteris, 60 arba 120 min. trukmė ir imituoto mokėjimo rezultatas. Įėjimui ir išėjimui - bilieto kodas bei veiksmo tipas. Laiką nustato serveris. Pavyzdys: vyrų grupė, 60 min., mokėjimas sėkmingas, laikas 10:00.

Išvestis: Kaina, bilieto kodas ir būsena arba aiški atmetimo priežastis.

Veikimo eiga:

1. Klientas pasirenka grupę ir apsilankymo trukmę.
2. Sistema pateikia kainą.
3. Sistema vienoje transakcijoje patikrina laisvas vietas ir patvirtina imituotą pirkimą. Jei vieta yra, išduodamas bilietas su QR kodu.
4. Įėjimo metu pradedamas skaičiuoti apsilankymo laikas.
5. Išėjimo metu patikrinama apsilankymo trukmė.
6. Sistema užbaigia apsilankymą arba apskaičiuoja priemoną. Po būsenų pokyčių perskaičiuojamas laisvų vietų skaičius.

Taisyklės arba sprendimo žingsniai

1. Vietų kontrolė. Kiekvienai grupei parduodamų vietų skaičius lygus 25. skaičiuojamos būsenos „aktyvus“ ir „laukiama priemokos“. Kai laisvų vietų nėra, bilietas neišduodamas, net jei kitoje grupėje vietų yra.
2. Kodo ir būsenos tikrinimas. Įėjimas galimas tik su galiojančiu QR kodu. Įėjimo metu rezervacija pakeičiama aktyviu apsilankymu ir įėjimo laikas įrašomas į duomenų bazę. Aktyviu, pasibaigusiu ar užbaigtu bilietu pakartotinai įeiti negalima.
3. Kaina ir laikas. Kaina parenkama pagal savaitės dieną, grupę, paslaugos laiką. Laikas skaičiuojamas nuo įėjimo. Jei išėjimo momentas ne vėlesnis už apmokėto laiko pabaigą, priemoka lygi 0 €. Kitu atveju priemoka = 2 € kas 15 minučių.
4. Priemokos užbaigimas. Išėjimo metu fiksuojama priemoka ir būsena „laukiama priemokos“. Darbuotojas paprašo sumokėti priemoną ir atidaro vartus rankiniu būdu.

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	Viduje 20 vyrų, 0 rezervacijų. Perkamas 60min. bilietas	Apmokamas bilietas. Įeinama pro vartus. Išėjimas iš baseino po 60min.	Kaina 6€. Po pirkimo: 20 viduje, 1 rezervacija, 4 laisvos. Įėjus: 21 viduje, 0 rezervacijų. Išėjus: 20 viduje, 5 laisvos. Priemoka 0€, bilietas užbaigtas.
Konfliktas dėl paskutinės vietos	Viduje 24 moterys, 0 rezervacijų. Dvi skirtingos pirkimo užklauskos vienu metu	Abi perka 60 min. bilietą.	Išduodamas tik 1 bilietas už 6€. Kita užklausa atmetama dėl vietų trūkumo, jos mokėjimas nepatvirtinamas. Galutinis rezultatas: 24 viduje, 1 rezervacija, 0 laisvų.

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Laiko riba	Įėjimas 10:00, trukmė 60 min. Kiekvienas variantas vykdomas atskirai.	Bandyti išeiti 11:00:00, 11:00:01, 11:15:00 ir 11:15:01.	Priemokos atitinkamai: 0€, 2€, 2€, 4€. Tik pirmu atveju apsilankymas iškart užbaigiamas. Kitais laukiama priemokos.
Pasibaigęs bilietas	Bilietas pirktas 10:00 ir iki 10:15 nepanaudotas.	Bandyti įeiti 10:15:00.	Įėjimas atmetamas. Bilietas pasibaigęs. Jo rezervuota vieta atlaisvinta. Lankytojų skaičius nedidėja.
Klaida	Duomenų bazėje nėra tokio QR kodo.	Pateikti negaliojantį kodą įėjimui	Rodoma „Bilietas nerastas“. Bilietų būsenos ir vietų skaičius nesikeičia.
Priemokos apmokėjimas	Viduje 25 vyrai. Bilieto - įėjimas 10:00, trukmė 60min.	Išeiti 11:16. Darbuotojui patvirtinti apmokėjimą 11:20	Užfiksuojama 4 € priemoka, iki patvirtinimo lieka 25 viduje. Patvirtinus: bilietas užbaigtas, 24 viduje, 1 laisva. Antras patvirtinimas skaičiaus nekeičia.

Jei modulis naudoja AI: Netaikoma.

4. Kokybės atributas

Pasirinktas atributas: Patikimumas, vertinamas pagal talpos ir apsilankymų duomenų vientisumą.

Kodėl svarbus šiai sistemai: Neteisingas skaičius leistų parduoti daugiau bilietų nei yra vietų arba nepagrįstai atmesti klientą.

Tikrinimo scenarijus ir sąlygos: Sistema bus tikrinama vietinėje aplinkoje su Python ir SQLite duomenų baze. Vienai grupei likus 1 laisvai vietai, du klientai vienu metu bandys nusipirkti bilietą. Bandymas bus kartojamas 100 kartų, prieš kiekvieną bandymą grąžinant tuos pačius pradinis duomenis. Taip pat bus tikrinama, kas nutinka du kartus pakartojus įėjimą, išėjimą ar apmokėjimo patvirtinimą.

Sėkmės kriterijus: Per kiekvieną iš 100 bandymų turi būti išduotas tik 1 bilietas. Užimtų ir rezervuotų vietų skaičius vienai grupei negali viršyti 25. Pakartojus tą patį veiksmą, neturi būti sukurtas papildomas bilietas ar dar kartą pakeistas lankytojų skaičius.

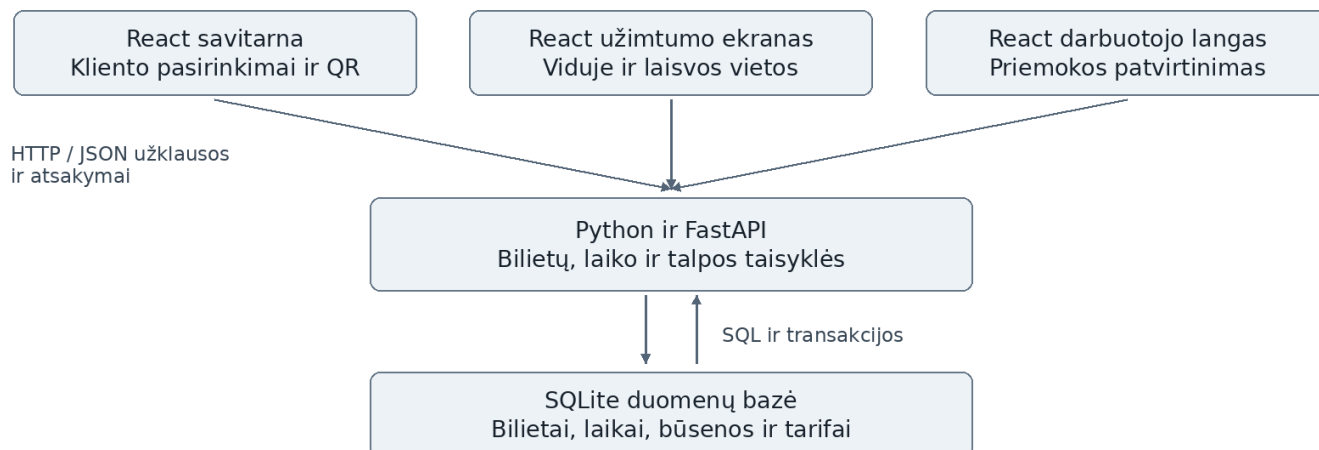
Numatytas projektavimo sprendimas: Vietos patikrinimas ir bilieto išdavimas bus atliekami vienoje duomenų bazės transakcijoje. Kol ji vyksta, kitas pirkimas turės palaukti. Kiekviena pirkimo užklausa turės savo unikalų numerį, kad ją pakartojus nebūtų išduotas antras bilietas. Sistema taip pat tikrins, ar su bilietu galima atlikti pasirinktą veiksmą. Užimtumas bus skaičiuojamas pagal duomenų bazėje esančius apsilankymų įrašus.

Kaip patikrinsiu vėlesniame etape: Automatiniai testai su užklausomis ir duomenų bazės patikra.

Sprendimo kaina arba ribojimas: Rašymo operacijos vykdomos paeiliui, todėl sprendimas skirtas nedideliame vietiniame prototipui.

5. Pradinė sistemos struktūra

Paprasta schema



Sistemos dalis	Atsakomybė
React	Savitarna priima pasirinkimus ir rodo bilietą. Viešas ekranas kas 3 s užklauskia užimtumą, klaidos atveju pažymi duomenis kaip neatnaujintus. Darbuotojo langas skirtas demonstracijai ir priemokai patvirtinti.
Python API ir pagrindinis modulis	API priima bei tikrina užklauskas. Atskiras logikos sluoksnis skaičiuoja kainą ir priemoką, tikrina būsenas, naudoja serverio laiką. Kainą ir laiką nustato serveris.
Duomenų bazė	SQLite saugo grupių limitus, tarifus ir apsilankymus: QR kodą, grupę, trukmę, išdavimo bei įėjimo laikus, būseną, priemoką ir mokėjimo požymį. Kelių įrašų pakeitimai atliekami transakcijomis.

Planuojamos technologijos ir pasirinkimo priežastys: Frontendui planuoju React ir JavaScript: tais pačiais komponentais galima kurti pasirinkimų formas, bilieto vaizdą ir užimtumo ekraną. Backendui - Python su FastAPI. React bendraus su juo JSON pranešimais. SQLite pasirinkta vietiniam prototipui, nes duomenų bazė saugoma viename faile ir nereikia atskiro serverio. Testams planuoju pytest.

6. AI panaudojimas

AI rengiant šį dokumentą

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
ChatGPT - pradinės sistemos struktūros schemos generavimas PNG formatu.	Sugeneruotą schemą, kurioje parodytos sistemos dalys ir jų ryšiai	Atskiros schemos versijos neatmečiau.	Palyginau schemą su sistemos aprašymu: ar parodyti React langai, Python backendas ir SQLite duomenų bazė.

Planuojamas AI naudojimas kuriant sistemą

AI planuoju naudoti kodo paaiškinimams, pradinių funkcijų ir testų pavyzdžiams. Siūlomą kodą tikrinsiu pagal taisykles, vykdysiu testus ir palyginsiu bibliotekų naudojimą su oficialia dokumentacija. Pagrindinės rizikos - klaidingas apvalinimas, neteisingi būsenų perėjimai ir nepatikrintas vienalaikiškumas.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
Patikslinti taisykles ir modelį	Aprašytos bilieto būsenos, sukurtos SQLite lentelės, nustatytos kainos ir paruošti testiniai duomenys.	Pirmiausia
Sukurti pagrindinį Python modulį	Veikiančios kainos, apsilankymo laiko, laisvų vietų ir bilieto galiojimo tikrinimo funkcijos. Paruošti jų testai.	Patikslinus taisykles.
Prijungti API ir duomenų bazę	Veikiantis bilieto pirkimas, įėjimo ir išėjimo registravimas, priemokos skaičiavimas bei užimtumo gavimas. Patikrinta, kas vyksta dviem klientams perkant vienu metu.	Sukūrus pagrindines funkcijas
Sukurti React langus	Veikianti savitarna, bilietas su QR kodu, užimtumo ekranas ir paprastas darbuotojo langas.	Paruošus backendą
Patikrinti ir paruošti demonstraciją	Patikrinta visa eiga nuo bilieto pirkimo iki išėjimo. Paruošti testavimo rezultatai, paleidimo instrukcija ir kodas įkeltas į GitHub.	Iki prototipo atsiskaitymo.

Būsimo prototipo veikimo scenarijus:

Demonstracijos pradžioje duomenų bazėje bus 24 vyrai ir 20 moterų, o rezervuotų vietų nebus. Užimtumo ekrane bus rodoma 1 laisva vieta vyrams ir 5 moterims. Savitarnoje pasirinksiu vyrų grupę ir 60 min. apsilankymą. Sistema parodys 6 € kainą. Patvirtinus imituotą mokėjimą 10:00, bus išduotas bilietas ir rezervuota paskutinė vyrų vieta. Kitas klientas vyrų grupės bilieto nusipirkti nebegalės, tačiau moterų grupėje vietų dar bus.

Įvedus bilieto kodą 10:02, bus užregistruotas įėjimas ir pradėtas skaičiuoti laikas. Viduje bus 25 vyrai, o rezervuotų vietų nebeliks. Apmokėtas laikas baigsis 11:02. Bandysiu išeiti 11:18, todėl laikas bus viršytas 16 min. ir sistema apskaičiuos 4 € priemoką. Darbuotojui patvirtinus apmokėjimą, apsilankymas bus užbaigtas. Viduje liks 24 vyrai ir atsiras 1 laisva vieta. Su tuo pačiu bilieto kodu dar kartą įeiti nebus galima.

Taip pat patikrinsiu išėjimą laiku, pavyzdžiui, 11:00. Tokiu atveju priemokos nebus ir sistema iškart leis išeiti. Demonstracijos metu naudosisu testinį laikrodį, kad galėčiau pakeisti laiką ir nereikėtų laukti visos valandos. Įprastai klientas sistemos laiko keisti negalės.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Pasirinktos taisyklės gali neatitikti realaus baseino darbo	Prieš programuodamas peržiūrėsiu kainas, 15 min. terminą įeiti ir nuo kada skaičiuojamas apsilankymo laikas. Prototipe naudosisu pavyzdines taisykles, kurias bus galima pakeisti atskiruose nustatymuose.
Du klientai gali nusipirkti paskutinę vietą arba pakartoti tą patį veiksmą	Vietos patikrinimą ir bilieto išdavimą atliksiu vienoje transakcijoje. Situaciją, kai du klientai perka vienu metu, tikrinsiu 100 kartų. Taip pat patikrinsiu pakartotinį pirkimą, įėjimą ir išėjimą.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Mokėjimų ir įrangos prijungimas gali užimti per daug laiko	Prototipe mokėjimai ir vartų veikimas bus imituojami. Pirmiausia sukursiu visą veikimo eigą su įvedamu bilieto kodu, o tada pridėsiu QR kodą ir sutvarkysiu langų išvaizdą.

Šaltiniai, jei naudojote

[1] FastAPI. Full Stack FastAPI Template. Python API ir React derinio pavyzdys.
<https://fastapi.tiangolo.com/project-generation/>

[2] FastAPI. SQL (Relational) Databases. SQLite naudojimo pavyzdys.
<https://fastapi.tiangolo.com/tutorial/sql-databases/>